PINTAR 4.0 · The owner rulings before implementation (D-118 batch 1)

6 October 2026. Prepared by the coordinator for Yazid and the Second Eye, as the closure register's §3 asks. **No product code has changed.** Each row reads left to right: existing canonical evidence → current behaviour → consequence → recommendation → implementation consequence. You rule row by row, not as a block.

What checking the evidence changed:

- **Two of the 13 are already decided.** Q1's two questions (the format, and whether a student's certificate shows the PINTAR ID) were ruled in D-080(2) and D-080(9). Q18 is your verbatim Q16 ruling of 5 Oct. The register had listed both as open. They need confirming and recording, not deciding again.
- **Two more only need recording.** QN2: D-082(a) already ruled the leaked-password check on. QN6: practice already differs from the evidence model's text.
- **So nine need a decision:** Q1's two leftovers, QN1, Q8, QN3, QN4, Q9, Q7, QN5, Q10 and QN7. QN7 is set from the measured cycle (§2), not from the old ~120-minute assumption.

1. The table

#	Existing canonical evidence	Current behaviour	Consequence	Recommendation	Implementation consequence
Q1 Certificate number (CR-01, CR-03)	D-080(2): «`<PINTAR ID>/<sequence>` (e.g. `A14CD7ST/00001`) is one indivisible display value». D-080(9): «C17: yes, a student's academic certificate may show their PINTAR ID, and with (2) it must». D-118(e): implementation history never becomes the ruling	Issues `<template prefix, or PINTAR/<SCHOOL>>/<YYYY>/<NNNNN>`, one sequence per school (20261126100000; 20261128100000 extended it on 6 Oct without reconciling D-080(2), the coordinator's error). Staging holds 3 synthetic certificates in that format; production holds none	The certificate Dr Gladys would see contradicts the design she approved. Every certificate issued before the fix keeps the wrong format for ever (sealed, INV-05)	**Already decided: confirm D-080(2) and (9).** Two leftovers to rule: (i) Studio's «Awalan nomor» has no place in the number: retire it with the layout picker (CR-06); (ii) the 3 staging certificates keep their numbers (sealed, synthetic)	BOB-48's number: a per-person 5-digit sequence under a lock on the person, never reused. ADM18-K01 and K02 restated, red then green. «Awalan nomor» removed from Studio and its save. GS-18 asserts the shape. One migration; no data rewritten

#	Existing canonical evidence	Current behaviour	Consequence	Recommendation	Implementation consequence
Q18 Wordmark (CR-03, CR-21, CR-56)	Your Q16 ruling, 5 Oct, verbatim on Daisy's branch (`ux/sertifikat/ronde-2/README.md, 05c7b51`): «The freeze is reopened solely to replace the superseded PINTAR SELALU product identity with the canonical PINTAR wordmark. This is a brand correction, not a redesign.» D-074(3) and D-076(b)(7) on `main` still say «PINTAR SELALU»	`main`'s DECISIONS has no record of it. The app's mark (`MerekPintar`) reads «PINTAR SELALU». The legal name «PT ZAYD PINTAR SELALU» in the footer and email is correct and stays	Anyone building from `main` would build the superseded identity onto the card and the certificate	**Already decided: record it verbatim** as a decision, superseding D-074(3) and D-076(b)(7). Unchanged: the legal name, and `verifikasi.pintarselalu.com` (D-080(8))	A DECISIONS row only. The wordmark lands in slices S1 (header), S4 (card, email) and S8 (certificate). It needs the font licence and outlined master, your action (CR-56)
QN1 Correct a staff email (CR-13)	J2.21 (P1): «correct a wrong email, without SQL». ADM-08. D-077 ruling (3), for the name: «platform admin only in 4.0 … a person's identity can span schools». D-054(a): an activation code goes only to the account's email. No ruling covers email	No path exists, not even an RPC. A mistyped staff email can only be fixed in SQL	Dr Gladys adds staff. One typo leaves a person who can never activate, and nothing on screen undoes it	**Before activation:** the school admin who provisioned the person corrects it; any outstanding code dies; audited with a reason. **After activation** (the email is then a credential channel): platform admin only, audited with a reason, with a notice to both the old and the new address. Students have no email (M2)	One RPC with two branches by activation state, red test first. A screen in the people list (slice S7). After-activation notices use the existing sender. One new GS step
Q8 Uji Cepat score (CR-27, CR-34)	J5.6 (P1): «Show the Uji Cepat score where designed». EDU-10. No design records where. D-082(d): a single Uji Cepat never issues a certificate. M3: completion and mastery are separate	The teacher sees a handed-in Uji Cepat question by question («Lihat per soal»). There is no class score and no learner score	The teacher ends a quick check without its one obvious number	**Both, on the teacher's Uji Cepat result only:** the class figure and each learner's score in one table. Never on the learner's mastery record or a certificate (M3, D-082(d))	Daisy designs it in slice S6 from existing session data. Probably a read only, with no schema change. Uji Cepat becomes a Level 3 step (CR-34)
QN3 Change an assignment's window (CR-28)	J5.9 / EDU-11, **P2**: «extend a deadline with a reason and an audit trail». D-093 T1 closed the direct table write	`extend_session_deadline` (one learner, audited, at most 3) exists with no screen. An assignment's window cannot be changed at all	A teacher who sets a wrong date must make a new assignment. Dr Gladys meets it only if she explores	**After go-live (P2, as frozen).** The one-learner extension gets its screen with the teacher's slice only if Daisy's S6 has room; otherwise P2 too	None now. If ruled into 4.0: an audited window-change RPC, a screen and a walk

#	Existing canonical evidence	Current behaviour	Consequence	Recommendation	Implementation consequence
QN4 Briefings (CR-37)	J7.5 / DEA-04 (P2): «honest states». D-061(#2), the queue being fed, is EC and done (BOB-39)	`/edukator/ringkasan` exists, but `open_briefing` has no caller, so it always reads «Belum ada…»	A surface that promises and never fills: a trust cost to a dean or head who opens it	**Hide the surface until 4.x** («honest states» is met by not offering it). The decision queue stays	Remove the entry points and the empty page from navigation (slice S6). Nothing else changes. Re-adding it later is a producer ticket
Q9 Misconceptions (CR-50, **mandatory before Gladys**)	D-063: definitions travel in packages, «usable by several schools with no cross-tenant ownership and no global-code collision». Your 2 Oct direction: identical national definitions should not be copied into every school thousands of times. J4.10 (EC)	`misconception_taxonomy` is keyed by the bare code, and nothing writes to it. Every chapter's review carries about 31 false «weak misconception» findings	A head of department acknowledges 31 untrue warnings per chapter. **A trust failure:** it is in the gate whatever you rule	**Canonical definitions owned by PINTAR's library, school-made definitions owned by the school**, both in one table keyed (owner, code), where the owner is the library or a school. National content links to the canonical rows; a school's own content to its own rows. It meets D-063(c) and your direction	D-063 M1–M5 adapted to two owners: the schema, the package's «Miskonsepsi» sheet, new problem codes and copy, the generators, the 20 workbooks regenerated, and Bab 1 re-uploaded as a revision. Tenancy tests. The largest item in the gate
Q7 PINTAR standard content (CR-51)	M6: «prepared once, governed centrally and distributed». J1.5 (EC). D-029: imports into the library are refused «for every role, Admin platform included». CF11: M6 amends D-029 but names no role (G2)	Distribution to schools is built (S26d). Nothing can author into the library	J1.5, an EC line, cannot be walked end to end	**The platform admin prepares and approves standard content**, through the same package and workbook import aimed at the library. D-029's refusal stays for every school role	One import target opened to the platform admin only, with its audit. One PINTAR chapter authored → distributed → assigned → learned as a walk
QN5 «Question Bank workflows» (CR-52)	D-050(a)/(d): «the Question Bank workflows intended for this phase, proved separately». J4.8 (EC). No record defines them	Studio → Bank Soal browses. Questions come only from workbooks	An EC line with no definition can never be proven	**Confirm the meaning:** browse, filter and review the school's questions with key and reason, and no authoring outside workbooks	No build expected. A walk of Bank Soal against that meaning; anything missing becomes an A item

#	Existing canonical evidence	Current behaviour	Consequence	Recommendation	Implementation consequence
Q10 «Not yet» band and grade labels (CR-47)	D-095(c): no Uji Penguasaan is «a distinct not-yet state». D-095(g) kept the calculation and the grade labels open. D-081: the school sets the pass mark. D-078 as amended: «Perlu Pendampingan» is descriptive and does not decide eligibility. Grade review, Alternatives 1–3	The not-yet band is never emitted. The predikat bands are fixed (Cukup ≥ 0.70), so at a 0.60 pass mark a certificate can read «Perlu Pendampingan»	A certificate that says the learner still needs help: the contradiction D-078 was written to avoid	**Alternative 1b:** «Cukup» starts at the assessment's own pass mark, so «Perlu Pendampingan» means below it and never reaches a certificate. A new scoring-policy version for new sessions only; sealed certificates keep their predikat. And emit the not-yet state	A scoring-policy version (`engine_version` bumped, D-060), the band calculation and the not-yet state, red then green. Daisy's tier words (MC-4). GS-17 walked again
QN2 Leaked-password check (CR-23)	D-033(c): off, «Revisit if passwords are ever typed by people». D-082(a): «leaked-password protection on»	On in staging's `auth-identitas.json` setting. Production is read at Gate 8	None for Gladys. The record looks contradictory	**Record only:** D-082(a) superseded D-033(c) by D-033(c)'s own condition (passwords are now typed)	A DECISIONS note. Production read at Gate 8
QN6 Level 3 cadence (CR-63)	EVIDENCE-MODEL: Level 3 «every merge to `main` that changes `src/` or `supabase/`; nightly; before a release»	Dispatch only, by a request file	The model claims evidence that is not produced	**Dispatch only:** after each slice that changes what the affected GS covers, before each gate, before a release. Correct the model's text	The text is corrected. Each run measured under D-119
QN7 The Actions allowance for this gate (CR-65)	D-091(e): CI's structural waste (local Supabase boots), fixes held for your timing; «every planted red must still go red». D-119: three numbers; less waste, not less assurance	**Measured (§2):** one authoritative cycle is compute 101 · queue 41 · estimate **123** minutes, 18 minutes' wait. The 25 database red cases are **99** of the 123, mostly booting a stack each	The gate needs about 18–20 cycles: **≈ 2,700–2,950 minutes without a structural fix**. At 600 a day, about four cycles a day	**First land S4** (the database red cases share a stack per group, each plant on a reset database), proven to keep every plant red. Then **≈ 2,100–2,250 minutes** for the whole gate. Allowance: **2,300**, reviewed after S4's first measured cycle. If S4 is declined: **2,950**	S4 in batch 0, one change with its own red-on-old proof and one measured cycle. Everything after is measured per run, and the allowance is re-set from that measurement, not from this forecast

2. QN7 from one measured authoritative cycle

The cycle: CI run 37450773138 on `0d80c36` (the D-119 controls in place), dispatched once. **37 of 37 jobs succeeded**, the verdict included («all 7 jobs ran and succeeded»). The strict selftest ran **597 tests: 597 passed, 0 failed, 0 skipped.** The guard's answer: «Not proven by an earlier authoritative run: CI runs in full». No job waited more than a minute for a runner.

Measure	This cycle (D-119)	Four full cycles before D-119 (5–6 Oct)
Compute (runner minutes)	**101**	100–105
Queue (minutes waiting)	41	26–33
Estimate (billable, each job rounded up)	**123**	117–126
Wall-clock	18.2 min	14.6–16.6 min
Jobs	37 (the guard and the verdict added)	35

What D-119 did and did not change. It removed the **waste around** a cycle:

- obsolete runs are superseded;
- a docs-only change is cited, not re-run (§3);
- nothing is queued while runners are starved;
- no job waits 20 minutes and dies at its timeout.

It did **not** make a cycle cheaper. A full proof costs the same: 123 against 117–126. Gating the expensive jobs behind lint, typecheck and build, and running at most 15 red cases at a time, add about 3 minutes of wall-clock and some queue time. The queue is not billed.

Where a cycle's minutes go:

Jobs	Estimate
25 database red cases, each booting its own local Supabase stack (about 2.5 of its 3 minutes)	**99**
Strict selftest	8
5 static red cases	5
Migrations (clean + upgrade path)	5
Lint, typecheck, build	2
Guard, baseline, verdict	4

The gate's need, built from measured units:

- one CI cycle = **123** (this run);
- one full Golden School walk = **44** (runs 37431278133 … 37434862707: 1 + 3 + 11 + 9 + 20);
- one end-to-end audit = **about 36** (job A 5, from 37267985806; job B 31 with C-12, from 37366709177).

Batch (register §2)	CI cycles	Walks and audits
0 Records and instruments	1–2	1 full GS (44)
2 Proof on today's code	1	1 full GS + 1 audit (80)
3 Behaviour (sealing and number; email; standard content; misconceptions)	4	the affected GS steps (about 60)

Batch (register §2)	CI cycles	Walks and audits
5 Propagation S1–S8	8	the affected GS per slice (about 160)
6 Full regression	—	1 full GS + 1 audit + U-PERF (about 90)
7 Your walkthrough's fixes	1–2	1 full GS re-pinned (44)
A red run re-run after its fix (about one in five)	+3	—
Total	**18–20**	**about 480**

	Without a structural fix	S4 first
Landing S4 (one change, about two cycles at 123)	—	about 250
The gate's cycles	18–20 × 123 = 2,214–2,460	18–20 × about 75 = 1,350–1,500
Walks and audits	about 480	about 480
The gate	**≈ 2,700–2,950**	**≈ 2,100–2,250**
Cycles a day inside 600	about 4	about 7

S4's saving is an estimate, not a measurement. Twenty-five boots become five, with a database reset (about 30 seconds) between plants: about 50 boot-minutes saved, less about 12 of resets, and less rounding over fewer jobs. That brings a cycle to **about 70–80**. The figure is re-measured on S4's first cycle, and the allowance follows the measurement. **S2** (group the five static red cases: about 3 minutes) and **S5** (cache the browser: under 1) are too small to justify touching the gate's machinery now.

Recommendation for QN7:

- (a) **S4 first, inside batch 0**, as one change with its own proof: every planted defect still turns its own gate red on a database reset before it (F01-R04 receives the same evidence), red on the old workflow where the new one would hide a plant.
- (b) An allowance of **2,300 estimated minutes for the gate**, reviewed after S4's first measured cycle and at each batch report.
- (c) The 600-a-day budget stays as the daily alarm. On a day of heavy work it may go red on cost: that is the record, and the waste alarm stays separate.

The bill. These are estimates: each job that ran, rounded up to the minute, as GitHub bills hosted runners. GitHub's per-run timing reports 0 for this repository, so the bill itself is on your billing page. I cannot see your plan's included minutes.

Today's spend on CE-8 itself:

Run	What	Compute	Estimate
37447512132	v1, full	105	127
37449744168	v2; superseded when the pull-request hole was found	15	22
37450178108	superseded when the alarm's false red was found	36	47
37450773138	the authoritative cycle	101	123

The two supersedes cost 69 estimated minutes. Each was cheaper than finishing a run on code about to change.

3. CE-8: the guards can never make CI falsely green or falsely proven

The rule. A CI run is evidence only when its conclusion is `success`. Two things accept evidence:

- the deploy gates (`deploy-vercel-production.yml`, `deploy-supabase-production.yml`): a `ci.yml` run on the exact commit with conclusion `success`;
- the guard's own reuse search: a `ci.yml` run with conclusion `success`.

So the proof is that nothing short of a complete, passing run can end in `success`.

Every outcome, enumerated:

	Situation	What the guard does	The run ends	Deployable?	Citable by a later run?
a	**Runners starved:** a job of another run has waited more than 10 minutes for a runner	Writes **DEFERRED · NOT PROVEN**, sets `proven=false`, cancels its own run before anything else is queued	`cancelled`	No	No
b	Starved, and the cancel does not take hold within 90 s	Exits 1	`failure` (and `verdict` fails: the guard's result is `failure`)	No	No
c	The guard cannot read GitHub (an error after its retries)	Exits 1	`failure`	No	No
d	**Code identical** to an authoritative run (branch only; push or dispatch; strict selftest `success`; at most 72 h; only inert documents differ)	Writes **ALREADY PROVEN by run X**, cancels its own run. Run X stays the evidence	`cancelled`	No (deploys come from `main`, where every run is in full)	No: X is cited, never this run
e	Code identical, and the cancel does not take hold	Exits 1	`failure`	No	No
f	Anything else: on `main`, code changed, no authoritative candidate	Writes «CI runs in full», `proven=false`	`success` only if `verdict` sees all seven jobs `success`; otherwise `failure`	Only if `success`	Only if `success`
g	Any job fails, is skipped or is cancelled in a full run	—	`verdict` fails → `failure` (or `cancelled`)	No	No
h	A newer run on the same branch supersedes it	—	`cancelled`	No	No

Why `success` means the full proof ran. GitHub counts a skipped job as harmless, so before `verdict` a run whose expensive jobs were skipped could still end `success`. That was the hole in v1. `verdict` needs all seven jobs (`reuse`, `check`, both migration jobs, the red cases, baseline parity, the strict selftest). It runs whenever the run is not cancelled (`!cancelled()`), and it fails unless every result is `success`, so a skipped job is a failure.

The four false-proven paths found and closed while proving it:

1. **A whole-folder rule:** v1 treated everything in docs/coordination, docs/reports and docs/deployment as unread. tsconfig and lint read the code files there (CE-5). Now only inert documents count.
2. **Renames:** `git diff` paired a renamed file, so moving code out of `src/` into a docs folder could hide its old path. Now `--no-renames` lists both paths.
3. **Skipped proof:** v1's ALREADY PROVEN skipped the jobs and let the run end `success`. Now it cancels, and `verdict` fails any skip.

4. **Pull request runs** (found 6 Oct, `d0d06cb`): such a run tests the merge of its head with the base, so its head was never proven on its own. Now only push and dispatch runs are cited.

Each is a CI-W02 assertion, red on the old guard and green on the new.

Demonstrated live, 6 Oct:

- **ALREADY PROVEN.** Commit `3b1c89b` changed six documents in docs/reports and nothing else. CI run 37453037724's guard wrote: «**ALREADY PROVEN** by CI run 37450773138 on `0d80c36` …, whose strict selftest ran and succeeded: this commit differs only in 6 unread document(s). This run cancels itself and is not evidence; run 37450773138 is.» The cancellation took hold 13 seconds later. The run ended `cancelled` after 30 seconds, at compute 0.6 · estimate 2 minutes against 123 for a full cycle. No job ran past the guard, and the monitor counts its 6 unstarted jobs as cancelled before a runner, not as waste.
- **Supersede.** Runs 37449744168 and 37450178108 were cancelled by the next dispatch on the branch (CE-2), as §2's spend table shows.

Checked against history: the 30 most recent successful CI runs (2 to 6 Oct) each have every job `success`, none skipped, and the strict selftest `success`. No earlier run is a false green the guard could cite.

What is not proven by a live run, and why: DEFERRED cannot be demonstrated safely. It needs a real runner shortage, and creating one would mean deliberately queueing work that cannot run, the waste D-119 forbids. Its proof is therefore the code path (it calls `cancelSelf()` before anything else), the cancel-or-exit-1 rule, the `verdict` job, and CI-W02, which executes `starved()` and reads that both outcomes call `cancelSelf()` and that `cancelSelf()` ends in `process.exit(1)`.